Digital Control: Sampling a Continuous Controller

Contents

1. Overview . . . 2
2. What the computer sees: sampling . . . 4
3. What the motors get: the D/A and the zero-order hold . . . 6
   - 3.1 Holding each command until the next one . . . 6
   - 3.2 The hold delays by half a sample . . . 8
4. From a formula to code: difference equations . . . 9
   - 4.1 The integral becomes a running sum . . . 9
   - 4.2 The PI controller as a loop of code . . . 10
   - 4.3 Three ways to add up the area: forward Euler, backward Euler and Tustin 12
   - 4.4 Which rule stays stable . . . 13
5. The z-transform: one symbol for one sample of delay . . . 15
   - 5.1 z to the minus one means one tick ago . . . 15
   - 5.2 The three rules in z . . . 16
   - 5.3 The robot in z, and poles inside the unit circle . . . 18
6. Choosing the sample rate . . . 23
7. Aliasing: a fast wave that looks slow . . . 24
   - 7.1 The wheel that seems to turn backwards . . . 24
   - 7.2 What aliasing does to the loop, and the anti-aliasing filter . . . 26
8. Where real stacks sample . . . 28
9. Summary . . . 29
10. Sources . . . 30
11. Key terms . . . 31

Where this fits:

- **Builds on:** PID control; Frequency response (gain and phase).
- **Compare with:** Transfer functions.

1. Overview

Key point: A computer runs a controller as a loop of code: at every tick of its clock it reads the sensors, computes one command and holds that command until the next tick. Run often enough, the code behaves like the formula; run too rarely, the hold's delay adds overshoot, the loop can lose its stability, and fast waves in the readings turn into slow false ones.

The chapter's robot can now hold its speed. Its speed controller is a PI controller, a PID controller (G-2965) without the derivative part: proportional control (G-2958) plus integral action (G-2963). The PI controller adds a force proportional to the speed error and a force that grows with the error's running total. Anti-windup (G-3078) stops that total growing while the drive is at its limit, and the gains can be tuned from a bump test with a derivative that does not kick. We keep the speed PI's gains throughout:

- the robot: mass 8 kg, rolling drag 2 N per m/s, drive force limited to ± 4 N;
- the proportional gain, 20 N of force per m/s of speed error;
- the integral gain, 10 N per m of accumulated error (per m/s of error held for 1 s).

With these gains the closed loop's poles (G-3000), the rates at which its own motion dies away, are -0.575 and -2.175 rad/s (notebook).

The PI controller is a formula in continuous time: its integral adds up the error at every instant. The robot's computer cannot do that: the computer can only add and multiply numbers, one step at a time, and it runs the speed loop 100 times a second, at 100 hertz (Hz, "times per second"). Between two runs it reads nothing and changes nothing. So two questions remain before the formula can drive the robot:

1. How do we write the formula as code that runs once per tick?
2. How often must that code run?

Figure 1 runs the same PI code at three rates. The robot cruises at 0.4 m/s and the command steps to 0.5 m/s at time 0. Watch the bottom panel: run twice a second, the robot's overshoot (G-2983), how far it passes the command before settling, is more than three times what the formula gives.

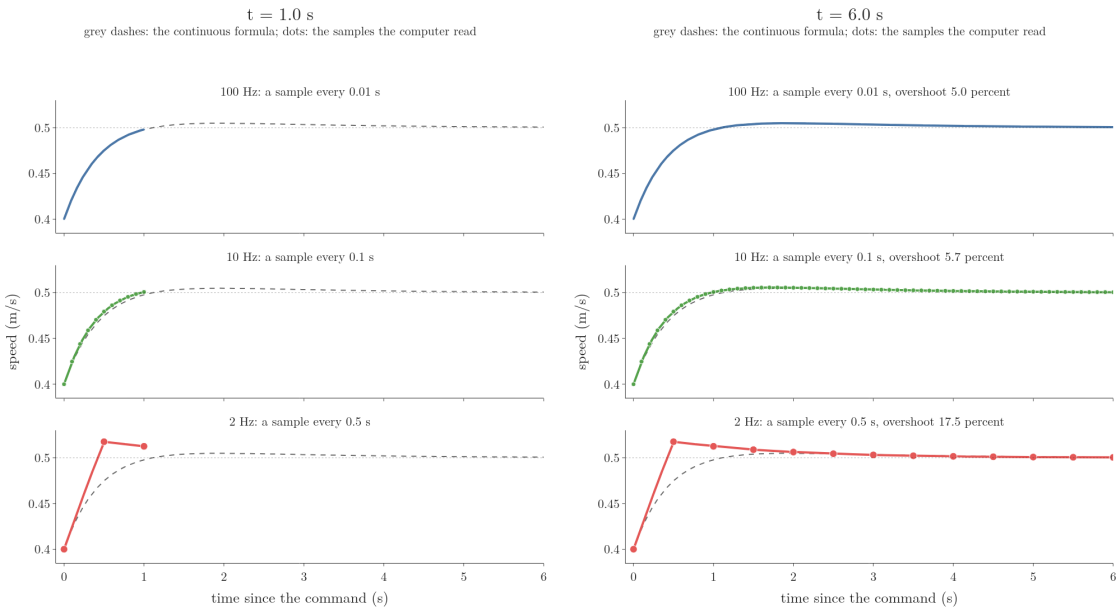


Figure 1: The PI speed controller run as code at 100 Hz (top, blue), 10 Hz (middle, green) and 2 Hz (bottom, red) after the command steps from 0.4 to 0.5 m/s, one frame per 0.1 s. Thick line: the robot's true speed; dots: the readings the computer took; grey dashes: the continuous formula's response. The last frame gives the overshoots: 5.0, 5.7 and 17.5 percent of the 0.1 m/s step, against 4.9 percent for the formula

The numbers (notebook):

- **100 Hz:** overshoot 5.0 percent of the step, against 4.9 percent for the formula;
- **10 Hz:** 5.7 percent;
- **2 Hz:** 17.5 percent, more than three times the formula's;
- **0.5 Hz:** the speed never settles; 20 s after the step it still swings between −0.21 and 1.2 m/s, held in only by the 4 N limit of the drive.

A second trouble hides in the readings. A worn wheel makes the speed reading ripple once per wheel turn, 1.59 times a second. Read twice a second, that ripple looks like a slow wave of 0.41 Hz, the controller chases it, and the robot really surges by 0.0119 m/s, nearly five times the 0.0025 m/s it shows at 100 Hz.

This Note turns the formula into code and explains both troubles:

- what the computer sees: **sampling** (G-3105) of the speed at each tick (Section 2);
- what the motors get: a force held constant between ticks, and why that hold acts like a delay of half a tick (Section 3);
- the formula as code: the integral becomes a running sum, added up by one of three rules (Section 4);
- the shorthand that turns code back into algebra, the **z-transform** (G-3114), and the test for stability it gives (Section 5);
- how to choose the sample rate (Section 6);
- why a fast wave can look slow, **aliasing** (G-3116), and how to stop it (Section 7);

- the rates real robot stacks run at (Section 8).

2. What the computer sees: sampling

> **Key point:** The computer reads the robot's speed only at the ticks of its clock, every sample period T (0.01 s, so 100 times a second); between ticks it knows nothing. Taking a signal's values at regular ticks is sampling, and an A/D converter turns each reading into a number.

The first part of the opening problem is that the computer sees the speed only at its ticks. A clock inside the robot's computer ticks at a fixed pace. At each tick the computer reads its sensors, works out a command and sends it out (FBS §8.5). A driver who looks at the speedometer only once every few seconds, and keeps the pedal still in between, drives the same way. Taking the values of a signal at regular instants, and only then, is **sampling** (G-3105) (NPTEL DC M1 L1). The time between two ticks is the **sample period** (G-3106), and the number of ticks per second is the **sample rate**. With:

- T, the sample period, in s: 0.01 s for the robot;
- f_s, the sample rate, in Hz (ticks per second).

$$f_s = 1/T$$

$$= 1/0.01 = 100 \text{ Hz}$$

Many sensors give a voltage, such as a gyro's rate signal or a motor's current sense. The device that turns that voltage into a number at each tick is the **analog-to-digital converter** (G-3107), A/D converter for short (FBS §8.5; NPTEL DC M1 L1). The robot's speed comes from its wheel encoders, which count marks on the turning wheel, and the speed is worked out from how the count changes between ticks (MR §11.1); the computer reads that estimate at each tick, as one number per tick.

What the samples show depends on the rate. A worn wheel makes the point clear. We take a wheel 1 mm out of round: its radius is 0.049 m on one side and 0.051 m on the other, instead of the 0.05 m the software assumes. The speed reading is the wheel's spin times 0.05 m, and a wheel that rolls without slipping (G-2290) spins at the robot's speed divided by its radius. At a steady 0.5 m/s the reading therefore swings once per turn:

$$\text{small side: } 0.5 \times 0.05/0.049$$

$$= 0.510 \text{ m/s}$$

$$\text{large side: } 0.5 \times 0.05/0.051$$

$$= 0.490 \text{ m/s}$$

The reading ripples by about 0.01 m/s either side of the true 0.5 m/s, although the robot's speed does not change. The wheel turns at the speed divided by the radius, in radians per second:

$$0.5/0.05 = 10 \text{ rad/s}$$

One turn is 2π radians, so the ripple repeats, in times per second:

$$10/(2\pi) = 1.59 \text{ Hz}$$

Figure 2 samples that reading at 100, 10 and 2 Hz. Watch the dots appear as the clock line sweeps across. At 100 Hz they trace the wave; at 10 Hz, about six dots per wave, they still follow it. At 2 Hz there are fewer than two dots per wave, and in the last frame the dots, joined up, show a slow wave that is not in the reading at all.

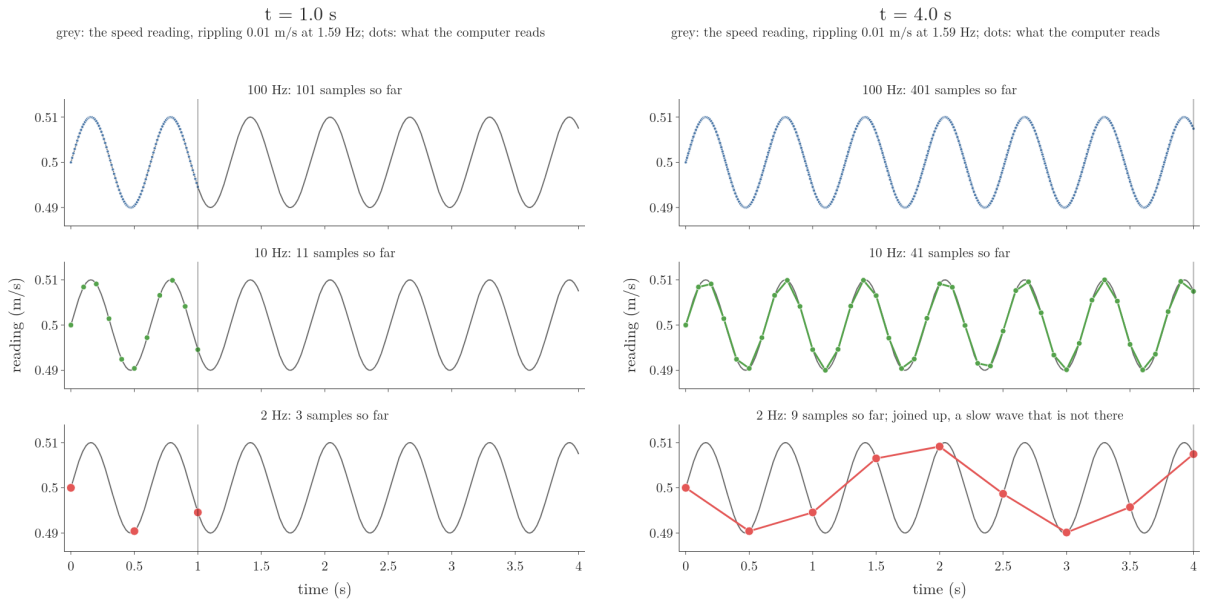


Figure 2: The speed reading of a robot cruising at 0.5 m/s on a wheel 1 mm out of round (grey): a ripple of 0.01 m/s, 1.59 times a second. A clock line sweeps from 0 to 4 s and a dot appears at every sample: 100 Hz (top, blue), 10 Hz (middle, green) and 2 Hz (bottom, red). The last frame joins the dots of each row: at 100 and 10 Hz they follow the ripple; at 2 Hz they trace a slow wave that is not there

So the samples are all the computer knows, and a slow rate can mislead it; Section 7 explains the false wave. Sampling is half of the loop. The other half is the command going back out, and the motors need a force at every instant, not only at the ticks.

3. What the motors get: the D/A and the zero-order hold

> **Key point:** The computer produces one force per tick; a D/A converter sends it out, and a zero-order hold keeps it constant until the next tick, so the motors get a staircase. On average the held value is half a sample period old, so the hold acts like a delay of $T/2$, 0.25 s at 2 Hz, which is the main reason the 2 Hz loop of Figure 1 overshoots.

3.1 Holding each command until the next one

The second part of the opening problem is on the way out: the computer has a new force only once per tick. Figure 3 draws the whole loop as the computer runs it, after FBS Fig. 8.14 and the CTM digital tutorial. Follow the signals round: the robot's smooth speed becomes numbers at the A/D converter, the computer turns each number into a force, and the D/A converter sends that force out. A purple clock drives both converters, so every reading and every command happens on a tick (CTM digital tutorial).

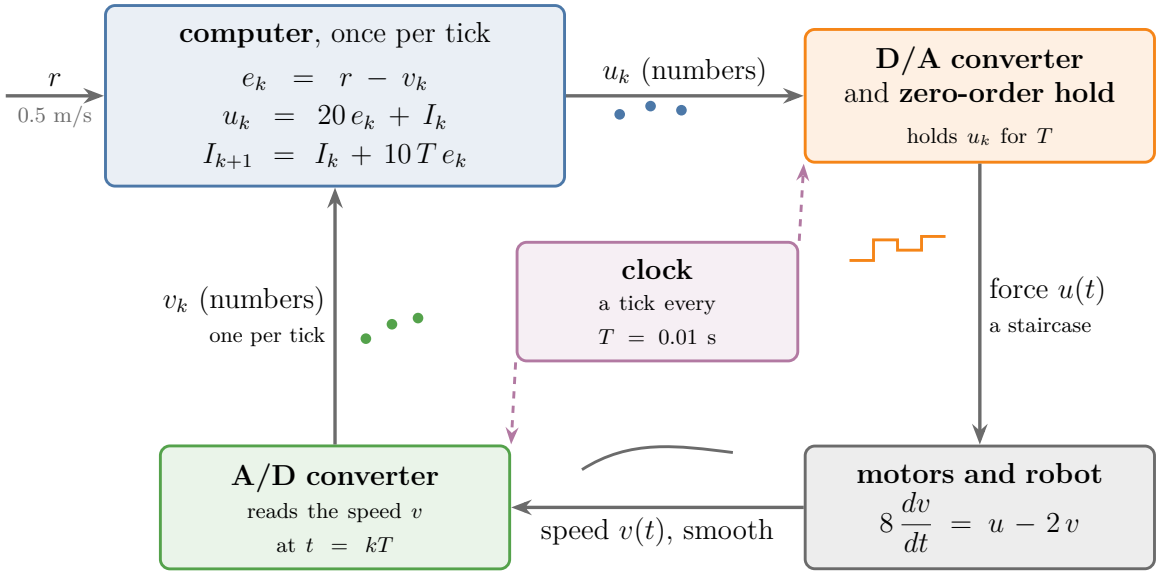


Figure 3: The speed loop as a computer runs it. A clock (purple) ticks every T = 0.01 s. At each tick the A/D converter (green) reads the robot's smooth speed v(t) and passes one number, v_k, to the computer (blue). The computer works out the error e_k = r − v_k against the command r = 0.5 m/s, the force u_k = 20 e_k + I_k and the next integral term I_k+1 = I_k + 10 T e_k. The D/A converter with a zero-order hold (orange) turns u_k into a force held constant until the next tick, a staircase, which drives the motors and robot (grey), 8 dv/dt = u − 2v. Small sketches on the wires: dots for numbers, a staircase for the held force, a smooth curve for the speed

The device that turns a number back into a voltage for the motor driver is the **digital-to-analog converter** (G-3108), D/A converter (FBS §8.5). The D/A converter is given a new number only at a tick, so it must decide what to send between ticks. The simplest answer is

to keep sending the last number until the next one arrives. Holding each value constant over its sample period is the **zero-order hold** (G-3109), "zero order" because it uses no slope, only the value itself (CTM digital tutorial; NPTEL DC M1 L4). The zero-order hold is the most common way to rebuild a signal from samples because it is simple and cheap (NPTEL DC M1 L4).

Figure 4 builds the hold at 2 Hz, one tick per frame, on the force the formula asks for during the speed step of Figure 1. Watch each orange dot: the D/A takes the formula's newest value and holds it flat for 0.5 s, while the grey curve keeps falling underneath.

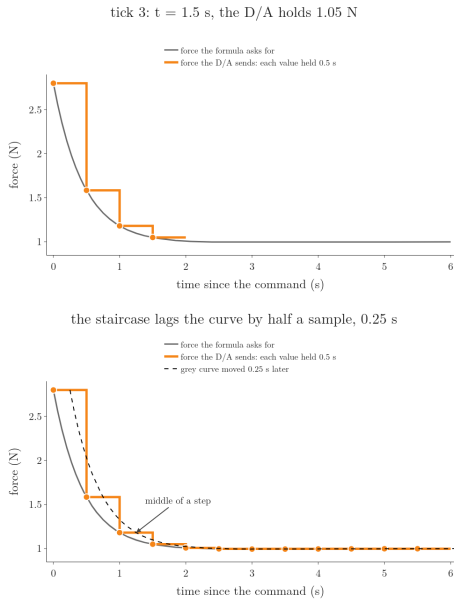


Figure 4: The zero-order hold at 2 Hz, one tick per frame. Grey: the force the continuous PI formula asks for after the speed step, from 2.8 N down to about 1 N. Orange: what the D/A converter sends; at each tick (dot) it takes the newest value and holds it for 0.5 s, so the force is a staircase. The last frame adds the grey curve moved 0.25 s later (black dashes): it runs through the middle of every step

In symbols, with:

- k, the tick number: 0, 1, 2 and so on;
- u_k, the force computed at tick k, in N: 2.8 N at tick 0 in Figure 4;
- $u(t)$, the force the motors get at time t, in N.

$$u(t) = u_k \text{ for } kT \leq t < (k+1)T$$

The faster the ticks, the closer the staircase lies to the smooth curve (NPTEL DC M1 L4). At 100 Hz each step lasts 0.01 s and is too short to see.

3.2 The hold delays by half a sample

The staircase in Figure 4 always trails the grey curve: each held value is already old when it is still being used. How old, on average? Over one hold of 0.5 s, the value's age grows evenly from 0 s at the tick to 0.5 s just before the next tick. An evenly growing quantity averages to the middle of its range:

$$\text{average age} = (0 + 0.5)/2$$

$$= 0.25 \text{ s}$$

The same for any sample period T:

$$\text{average age} = (0 + T)/2$$

$$= T/2$$

The last frame of Figure 4 checks this: the grey curve moved 0.25 s later passes through the middle of every step. The frequency response of the zero-order hold says the same exactly: its phase (G-3022), the lag of a wave passing through it, is the lag of a pure delay of $T/2$ (NPTEL DC M1 L4).

Why a delay adds overshoot. A delay of D seconds shifts a wave that turns at ω radians per second by ω times D radians (time delay (G-3012)). The loop's phase margin (G-3045) is how much extra lag the loop can take, at the rate where its loop gain is 1, before it oscillates; that rate is the gain crossover frequency (G-3042). Our speed loop crosses gain 1 at 2.536 rad/s with a phase margin of 84.5° (notebook). At 2 Hz, with:

- ω_c, the gain crossover frequency, where the loop gain is 1, 2.536 rad/s;
- $T/2$, half a sample period, 0.25 s.

$$\omega_c \times T/2 = 2.536 \times 0.25$$

$$= 0.634 \text{ rad} = 36.3^\circ$$

$$84.5^\circ - 36.3^\circ = 48.2^\circ$$

The hold eats over two fifths of the margin, and a smaller margin means a more oscillatory response. At 10 Hz half a sample, 0.05 s, costs 7.3°; at 100 Hz, 0.005 s costs 0.7°. As a check, the continuous loop with a pure delay of 0.05 s added overshoots by 5.4 percent, close to the 5.7 percent of the loop run at 10 Hz (notebook). At 2 Hz the match is rougher, 23.9

percent with a pure 0.25 s delay against 17.5 percent for the loop run at 2 Hz, because the half-sample delay is a picture that holds best when a sample is short (the hold's accuracy depends on the sample rate, NPTEL DC M1 L4); both still lie far above the formula's 4.9 percent. So the extra overshoot in Figure 1 comes from the hold's delay. The hold explains what the motors get; the next question is what the computer computes in between, since its integral cannot add up the error at every instant.

4. From a formula to code: difference equations

> **Key point:** In code the integral becomes a running sum: at each tick add the error times the sample period T. The rule that gives the next value from the present ones is a difference equation. Three rules add up the strips under the error differently (forward Euler, backward Euler, Tustin); all agree when T is small, and forward Euler blows up when a filter inside the controller has a time constant shorter than half a sample period.

4.1 The integral becomes a running sum

The first opening question, how to write the formula as code, comes down to the integral term: the formula adds up the error at every instant, and the computer has the error only at the ticks. With:

- $u(t)$, the force sent to the motors, in N: 2.8 N just after the step;
- $e(t)$, the speed error, the command minus the speed, in m/s: 0.1 m/s just after the step;
- k_p, the proportional gain, 20 N per m/s;
- k_i, the integral gain, 10 N per m;
- $I(t)$, the integral term, k_i times the area under the error up to time t, in N: 0.8 N while cruising at 0.4 m/s, the force that balances the drag.

$$u(t) = k_p\, e(t) + I(t)$$

$$I(t) = k_i \int_0^t e\, dt + 0.8$$

The area under the error over one tick is a strip one sample period T wide. If T is short, the error barely changes across the strip, so the strip is close to a rectangle as tall as the error at its start, e_k. Adding one such strip per tick turns the integral into a running sum. The running sum is the Euler step (G-2299) applied to the integral: hold the rate fixed for one step and add rate times step (FBS §11.5, eq. 11.17). With:

- e_k, the error read at tick k, in m/s: 0.1 m/s at tick 0;
- I_k, the integral term at tick k, in N: 0.8 N at tick 0;
- T, the sample period, 0.1 s at 10 Hz.

At tick 0:

$$I_1 = I_0 + k_i\,T\,e_0$$

$$= 0.8 + 10 \times 0.1 \times 0.1$$

$$= 0.9 \text{ N}$$

The same for any tick:

$$I_{k+1} = I_k + k_i\,T\,e_k$$

An equation that gives a sequence's next value from its present and past values, such as this one, is a **difference equation** (G-3110) (NPTEL DC M1 L2). Difference equations are what a computer can run: each tick needs only a multiplication and an addition (FBS §8.5).

4.2 The PI controller as a loop of code

With the running sum, the whole controller is a few lines. The order follows FBS §11.5: wait for the tick, read, compute, send the force out at once, and only then update the integral, so the force leaves as soon as possible after the reading.

```
T = 0.01                    # sample period, s (100 Hz)
I = 0.8                     # integral term, N
while True:
    wait_for_tick()         # the clock
    v = read_speed()        # A/D converter
    e = 0.5 - v             # error, m/s
    u = 20 * e + I          # force, N
    send_force(u)           # D/A, held until the next tick
    I = I + 10 * T * e      # update after sending
```

Figure 5 runs this code at 10 Hz, one tick per frame, with the four lines of each tick and their numbers at the top. Here the A/D converter, which turns each reading into a number, rounds the reading to 0.0001 m/s, so every number shown is exact. Watch the dots: each reading sets one orange step of the force. At 10 Hz the integral gain times the sample period is 1:

$$k_i T = 10 \times 0.1 = 1$$

So the integral term changes by exactly the error at every tick, and at tick 10, just past the command, it starts to shrink.

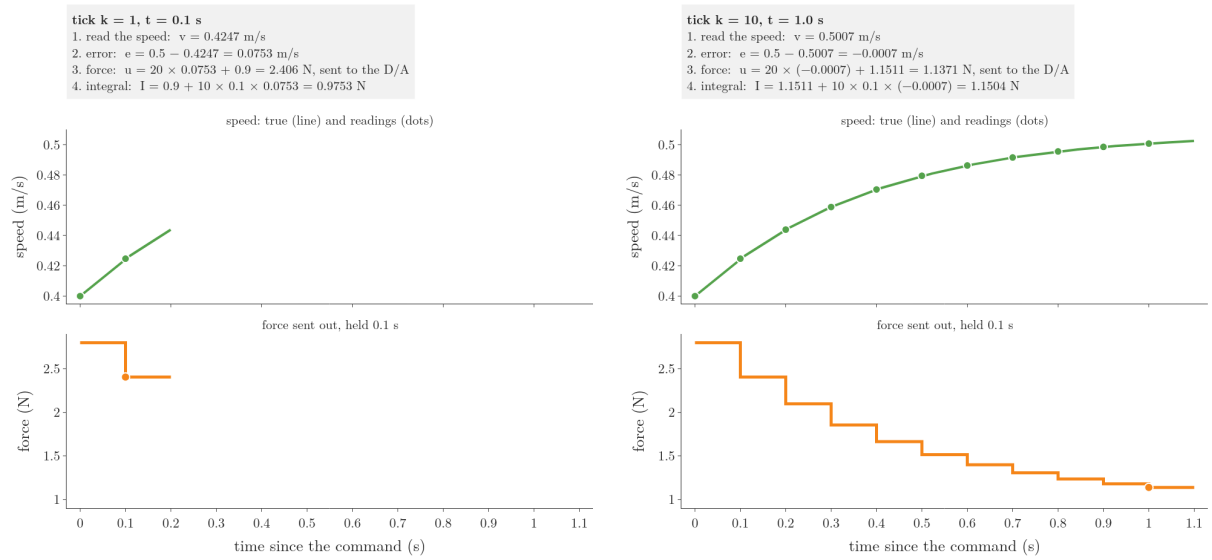


Figure 5: The PI controller as code at 10 Hz, one tick per frame, ticks 0 to 10. Top: the tick's four lines with its numbers (read the speed, the error, the force sent to the D/A, the new integral term). Middle: the robot's true speed (line) and the readings (dots). Bottom: the force sent out, each value held 0.1 s. Tick 1 reads 0.4247 m/s and sends 2.406 N; tick 10 reads 0.5007 m/s, just past the command, and sends 1.1371 N

The first two ticks by hand. Tick 0 reads the cruise speed, 0.4 m/s:

$$e_0 = 0.5 - 0.4 = 0.1$$

$$u_0 = 20 \times 0.1 + 0.8 = 2.8 \text{ N}$$

$$I_1 = 0.8 + 10 \times 0.1 \times 0.1 = 0.9$$

The 2.8 N is held for 0.1 s, and the robot speeds up to 0.4247 m/s. Tick 1:

$$e_1 = 0.5 - 0.4247 = 0.0753$$

$$u_1 = 20 \times 0.0753 + 0.9$$

$$= 1.506 + 0.9 = 2.406 \text{ N}$$

$$I_2 = 0.9 + 10 \times 0.1 \times 0.0753$$

$$= 0.9 + 0.0753 = 0.9753$$

Real controllers are written the same way. In ros2_control, `Pid::compute_command` in control_toolbox's `pid.cpp` first sums the proportional, integral and derivative terms into the command and only then adds `dt * i_gain * error` to the integral term, the order above. Apollo's `PIDController::Control`, in the `pid_controller.cc` of its control module, instead adds `error * dt * ki` to the integral before it computes the output. Both take the time step `dt` as an argument at every call, so a late tick is integrated with its true length. The two orders are two different rules for the area, which the next section compares.

4.3 Three ways to add up the area: forward Euler, backward Euler and Tustin

The strip of Section 4.1 takes the height of the error at its start. When the error is falling, that height is too tall for most of the strip. Figure 6 shows the size of the mistake at 2 Hz, on the error after the speed step, and two other rules. Watch the area each one keeps in its title, one strip per frame.

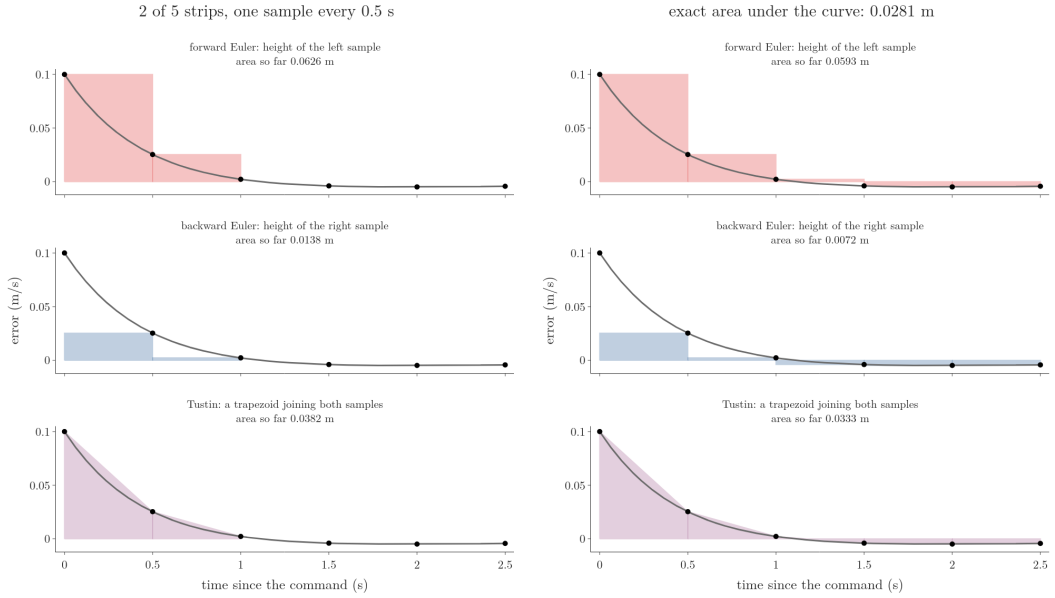


Figure 6: The area under the speed error after the step (grey), added up with one sample every 0.5 s, one strip per frame. Top (red): forward Euler, each strip as tall as its left sample. Middle (blue): backward Euler, as tall as its right sample. Bottom (purple): Tustin, a trapezoid joining both samples. The last frame gives the exact area from 0 to 2.5 s, 0.0281 m, against 0.0593 m, 0.0072 m and 0.0333 m

Turning a continuous controller into a difference equation is **discretisation** (G-3111). The three rules, with A_k the area added up to tick k, in m, and the error samples e_k and e_{k+1} at the two ends of the strip (NPTEL DC M1 L2; FBS §11.5):

- **forward Euler**, the Euler step above: the strip as tall as its left sample;

$$A_{k+1} = A_k + T\,e_k$$

- **backward Euler** (G-3112): the strip as tall as its right sample;

$$A_{k+1} = A_k + T\,e_{k+1}$$

- **Tustin's method** (G-3113), also called the trapezoidal rule or the bilinear transform: a trapezoid joining the two samples;

$$A_{k+1} = A_k + T\,(e_k + e_{k+1})/2$$

On the first strip of Figure 6, from 0.1 m/s at 0 s to 0.0253 m/s at 0.5 s:

$$\text{forward: } 0.5 \times 0.1 = 0.05$$

$$\text{backward: } 0.5 \times 0.0253 = 0.01265$$

$$\text{Tustin: } 0.5 \times 0.1253/2 = 0.031325$$

Over the five strips to 2.5 s, against the exact area 0.0281 m (notebook):

Rule	Area (m)	Error
forward Euler	0.0593	111 percent too much
backward Euler	0.0072	74 percent too little
Tustin	0.0333	19 percent too much

The trapezoid follows a sloping curve best, because it uses both ends of the strip. At 2 Hz the rules differ a lot; at 100 Hz the strips are 0.01 s wide and all three agree.

4.4 Which rule stays stable

Accuracy is not the only difference: one rule can make a stable controller unstable. Controllers often contain a low-pass filter, such as the derivative filter (G-3101) that smooths the derivative term. A low-pass filter with time constant T_f follows a step of its input, reaching 63 percent of the step after T_f. We take a fast one, $T_f = 0.02$ s, and run it at 10 Hz, so its time constant is shorter than the sample period. With:

- x_k, the filter's input at tick k: a step to 1 at tick 0;
- y_k, its output at tick k, starting at 0;
- T, the sample period, 0.1 s;
- T_f, the filter's time constant, 0.02 s.

The continuous filter changes its output at the rate (input − output) / T_f. Forward Euler holds that rate for one tick, so the remaining gap to the input changes by a fixed factor each tick:

$$y_{k+1} = y_k + (0.1/0.02)(1 - y_k)$$

$$1 - y_{k+1} = (1 - y_k) - 5\,(1 - y_k)$$

$$= (1 - 5)(1 - y_k)$$

$$= -4\,(1 - y_k)$$

The same in symbols:

$$1 - y_{k+1} = (1 - T/T_f)(1 - y_k)$$

Each tick multiplies the gap by -4: the output goes 0, 5, -15, 65, -255 and blows up. Forward Euler fails like this whenever the factor is below -1, that is when T is more than twice T_f. Backward Euler uses the new input and the new output in the rate:

$$y_k = \frac{T_f\, y_{k-1} + T\, x_k}{T_f + T}$$

The same steps as for forward Euler show that each tick multiplies the gap by:

$$T_f/(T_f + T) = 0.02/0.12$$

$$= 1/6$$

The factor $T_f/(T_f + T)$ lies between 0 and 1 for every sample period, so backward Euler is stable at any rate (FBS §11.5). Tustin averages the old and the new rate, and the same steps show that it multiplies the gap each tick by:

$$\frac{2T_f - T}{2T_f + T} = \frac{0.04 - 0.1}{0.04 + 0.1}$$

$$= -0.43$$

Its size is below 1, so Tustin is stable too, but the sign flips at every tick, so its output overshoots and rings. Figure 7 runs all three, one tick per frame.

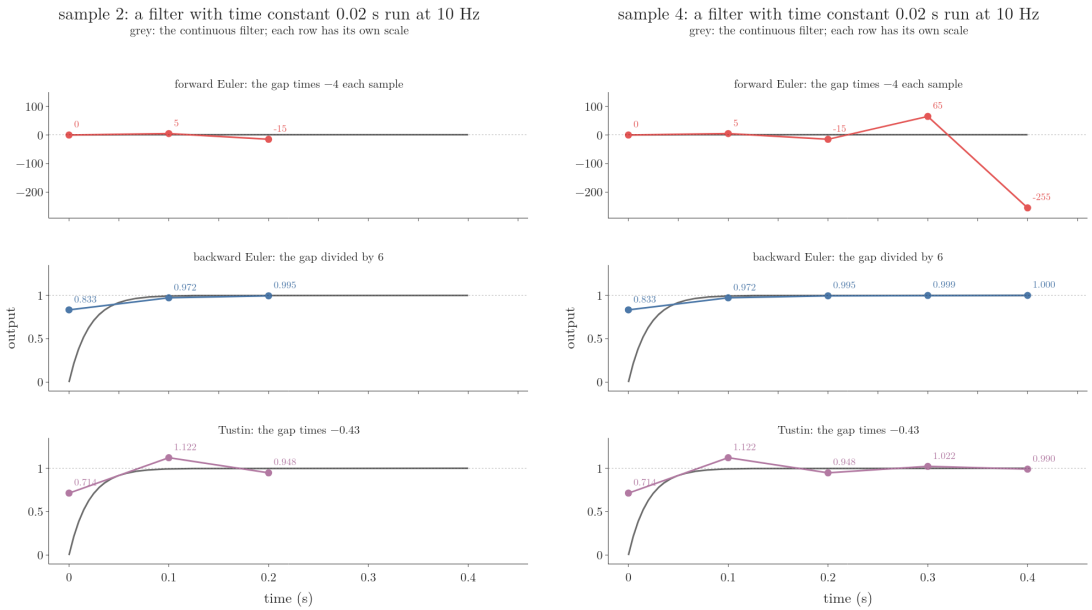


Figure 7: A low-pass filter with time constant 0.02 s, fed a step of 1 and run at 10 Hz, one tick per frame; grey is the continuous filter, already at 0.993 after 0.1 s. Top (red): forward Euler multiplies the gap to 1 by −4 each tick: 0, 5, −15, 65, −255. Middle (blue): backward Euler divides it by 6: 0.833, 0.972, 0.995. Bottom (purple): Tustin multiplies it by −0.43: 0.714, 1.122, 0.948. The top row has its own scale, hundreds of times larger

The gap factors explain why FBS §11.5 writes the integral with forward Euler but the derivative filter with backward Euler: the integral has no time constant to outrun, and a fast filter does. We now have the controller as code. To see why the 2 Hz loop of Figure 1 overshoots and the 0.5 Hz loop never settles, we need the whole loop, robot and code together, in one piece of algebra.

5. The z-transform: one symbol for one sample of delay

> **Key point:** Writing z^{-1} for "the same signal, one tick earlier" turns difference equations into algebra, as the Laplace variable s does for differential equations. The sampled loop then has poles in z, and it is stable only if every pole lies inside the circle of radius 1: at 2 Hz the speed loop's poles are 0.706 and 0.002, inside; below 0.89 Hz one lies outside.

5.1 z to the minus one means one tick ago

To find why the 2 Hz loop overshoots and the 0.5 Hz loop never settles, we must combine two difference equations, the robot's and the controller's. Each relates a signal to itself one tick earlier, and writing out "one tick earlier" every time makes long equations. A short symbol

helps. For a signal sampled at ticks 0, 1, 2 and so on, we write z^{-1} in front of it to mean "the same signal, delayed by one tick". The error samples of Figure 5, delayed by one tick:

$$e: \quad 0.1,\ 0.0753,\ 0.0561, \ldots$$

$$z^{-1}e: \quad 0,\ 0.1,\ 0.0753, \ldots$$

The bookkeeping behind the symbol is the **z-transform** (G-3114): it writes a sequence $f_0, f_1, f_2, \ldots$ as one expression, each sample multiplied by z^{-k} for its tick number k (NPTEL DC M2 L1):

$$F(z) = f_0 + f_1 z^{-1} + f_2 z^{-2} + \ldots$$

For the sequence 1, 0.5, 0.25:

$$F(z) = 1 + 0.5\, z^{-1} + 0.25\, z^{-2}$$

Delayed by one tick, the sequence is 0, 1, 0.5, 0.25, whose transform is every term moved one power of z^{-1} along:

$$0 + 1\, z^{-1} + 0.5\, z^{-2} + 0.25\, z^{-3}$$

$$= z^{-1}(1 + 0.5\, z^{-1} + 0.25\, z^{-2})$$

$$= z^{-1}F(z)$$

So a delay of one tick is a multiplication by z^{-1} (NPTEL DC M2 L1), just as, in the continuous world, the Laplace transform turns a derivative into a multiplication by s (transfer functions (G-2999)). The two are linked by $z = e^{sT}$ (NPTEL DC M2 L1), which Section 5.3 uses.

5.2 The three rules in z

With z^{-1}, each rule of Section 4.3 becomes a fraction in z that stands where the continuous integrator, a division by s, stood. With:

- A, the transform of the area added up so far, in m;
- E, the transform of the error samples, in m/s;
- T, the sample period, 0.1 s here, so half of it is 0.05 s.

Tustin's rule at $T = 0.1$ s, then the same rule in transforms:

$$A_k = A_{k-1} + 0.05\,(e_k + e_{k-1})$$

$$A = z^{-1}A + 0.05\,(E + z^{-1}E)$$

$$A\,(1 - z^{-1}) = 0.05\,(1 + z^{-1})\,E$$

$$\frac{A}{E} = 0.05\,\frac{1 + z^{-1}}{1 - z^{-1}}$$

$$= 0.05\,\frac{z + 1}{z - 1}$$

The same steps for any T:

$$\frac{A}{E} = \frac{T}{2}\,\frac{z + 1}{z - 1}$$

The continuous integrator is $1/s$, with s the Laplace variable. Setting $1/s$ equal to Tustin's fraction gives the substitution Tustin makes for s:

$$s \to \frac{2}{T}\,\frac{z - 1}{z + 1}$$

The same steps turn forward Euler's area into $T/(z-1)$ times the error and backward Euler's into $Tz/(z-1)$ times the error.

Now the PI controller at 10 Hz, discretised with Tustin. With $C(z)$, the controller's force divided by its error, in N per m/s:

$$C(z) = 20 + 10 \times 0.05\,\frac{z + 1}{z - 1}$$

$$= \frac{20\,(z - 1) + 0.5\,(z + 1)}{z - 1}$$

$$= \frac{20.5\,z - 19.5}{z - 1}$$

With U the transform of the force, the controller says $U = C(z)\,E$. Multiplying out and dividing by z turns it into code:

$$U\,(z-1) = E\,(20.5\,z - 19.5)$$

$$U\,(1 - z^{-1}) = E\,(20.5 - 19.5\,z^{-1})$$

$$u_k = u_{k-1} + 20.5\,e_k - 19.5\,e_{k-1}$$

Check at easy values: an error of 0.1 m/s that never goes away raises the force each tick by the integral's share, as it should:

$$(20.5 - 19.5) \times 0.1 = 0.1 \text{ N}$$

$$k_i\,T\,e = 10 \times 0.1 \times 0.1 = 0.1 \text{ N}$$

Libraries do this conversion: MATLAB's `c2d(C, 0.1, 'tustin')`, or, in Python, SciPy's `cont2discrete` with `method='bilinear'`, which returns the numerator 20.5, −19.5 over 1, −1 (notebook).

5.3 The robot in z, and poles inside the unit circle

The robot also needs a description at the ticks. Between two ticks its force is held constant, so its speed follows the first-order response: it moves towards the speed that force can hold, force divided by drag, with the time constant (G-2988), the time to cover 63 percent of the gap, equal to mass divided by drag:

$$8/2 = 4 \text{ s}$$

After one tick of $T = 0.5$ s, the gap to that target has shrunk by the factor:

$$e^{-0.5/4} = 0.8825$$

With:

- v_k, the speed at tick k, in m/s;
- u_k, the force held from tick k, in N; the target speed is $u_k/2$.

$$v_{k+1} = u_k/2 + (v_k - u_k/2) \times 0.8825$$

$$= 0.8825\,v_k + 0.05875\,u_k$$

Check at steady state, 0.5 m/s held by the drag force of 1 N:

$$0.8825 \times 0.5 + 0.05875 \times 1 = 0.5$$

The sampled description is exact, not an approximation, because the force really is constant between ticks: it is the zero-order-hold equivalent that CTM computes with `c2d(sys, Ts, 'zoh')` (CTM digital tutorial; FBS §8.5). For any sample period, with:

- $m = 8$ kg, the mass, and $b = 2$ N per m/s, the drag;
- a, the factor by which the gap shrinks each tick, 0.8825 at 2 Hz;
- g, the speed gained per newton held for one tick, 0.05875 m/s per N at 2 Hz.

$$a = e^{-bT/m}$$

$$g = (1 - a)/b$$

$$v_{k+1} = a\, v_k + g\, u_k$$

What a pole in z means. With no force, the robot's speed is multiplied by 0.8825 every tick: 1, 0.8825, 0.779 and so on. The speed dies away because 0.8825 is less than 1. In transforms, with V and U the transforms of the speed and the force:

$$V = z^{-1}(0.8825\, V + 0.05875\, U)$$

$$\frac{V}{U} = \frac{0.05875}{z - 0.8825}$$

The value of z that makes the bottom of this fraction zero, 0.8825, is the pole of the sampled robot: the factor that multiplies its own motion at every tick. The pole 0.8825 is the continuous pole $s = -0.25$ sent through the link between the two worlds (NPTEL DC M2 L1):

$$z = e^{sT}$$

$$= e^{-0.25 \times 0.5} = 0.8825$$

A pole can also be negative or complex. A negative pole flips the signal's sign at every tick, so the signal jumps up and down from one tick to the next; a complex pole shrinks the signal and turns it a little at every tick, a slower swing. We draw each pole as a point in the plane of complex numbers, the z-plane, real part across and imaginary part up. Its distance from the centre is the factor by which the signal shrinks or grows each tick:

- inside the circle of radius 1, the signal shrinks: the loop is stable;
- on the circle, it neither shrinks nor grows: it swings for ever;
- outside, it grows: the loop is unstable.

The circle of radius 1 around the centre of the z-plane is the **unit circle** (G-3115), and a sampled loop is stable exactly when all its poles lie inside it (NPTEL DC M3 L1; CTM digital tutorial).

The poles of the speed loop. At 2 Hz the integral term grows each tick by $k_i T$ times the error, with:

$$k_i T = 10 \times 0.5 = 5$$

With the command set to 0, since it does not move the poles, the robot's equation, the controller's force and its integral term at each tick are:

$$v_{k+1} = 0.8825\, v_k + 0.05875\, u_k$$

$$u_k = -20\, v_k + I_k$$

$$I_{k+1} = I_k - 5\, v_k$$

Putting the force into the robot's equation lowers the factor on the speed, here below zero:

$$0.05875 \times 20 = 1.175$$

$$0.8825 - 1.175 = -0.2925$$

$$v_{k+1} = -0.2925\, v_k + 0.05875\, I_k$$

The negative factor says that the proportional push alone, held for half a second, overcorrects: it removes more speed error than there was. In transforms, each "next tick" becomes a multiplication by z:

$$(z + 0.2925)\, V = 0.05875\, I$$

$$(z - 1)\, I = -5\, V$$

Multiplying the first line by $(z - 1)$ and using the second:

$$(z + 0.2925)(z - 1)\,V = -0.29375\,V$$

$$z^2 - 0.7075\,z - 0.2925 + 0.29375 = 0$$

$$z^2 - 0.7075\,z + 0.00125 = 0$$

The two poles are 0.706 and 0.002 (notebook). Both lie inside the unit circle, so the 2 Hz loop is stable, but neither sits where the continuous design puts it: its poles, -0.575 and -2.175 rad/s, sent through $z = e^{sT}$ land at 0.750 and 0.337 (notebook). A loop's response follows its poles, so the moved poles give the different response that Figure 1 shows as extra overshoot.

The same steps in symbols, for any T, with the gains $k_p = 20$ and $k_i = 10$:

$$(z - (a - g\,k_p))(z - 1) + g\,k_i T = 0$$

At $T = 2$ s (0.5 Hz) the robot's factors are $a = 0.6065$ and $g = 0.19673$ (notebook), and the integral's share is as large as the proportional gain:

$$k_i T = 10 \times 2 = 20 = k_p$$

So in the last number of the equation the two shares cancel, leaving a:

$$a - g\,k_p + g\,k_p = a$$

Multiplying out the brackets gives the number in front of z with a minus sign:

$$g\,k_p = 0.19673 \times 20 = 3.9346$$

$$a - g\,k_p + 1 = 0.6065 - 3.9346 + 1$$

$$= -2.3281$$

$$-(a - g\,k_p + 1) = 2.3281$$

$$z^2 + 2.3281\,z + 0.6065 = 0$$

The poles are −0.299 and −2.029 (notebook). The second lies outside the unit circle: each tick multiplies the swing by −2.03, flipping it and doubling it, and only the drive's 4 N limit stops it growing, as the overview found. Between 2 Hz and 0.5 Hz, a pole leaves the circle through −1 when the sample period reaches 1.12 s, a rate of 0.89 Hz (notebook).

Figure 8 lets the sample period grow from 0.01 s to 2 s. Watch the two red poles: both start near 1, on the hollow circles where the continuous poles land; as the period grows, both slide left along the real axis, and the faster one passes 0, then −1 at $T = 1.12$ s, while the step response on the right swings harder and harder.

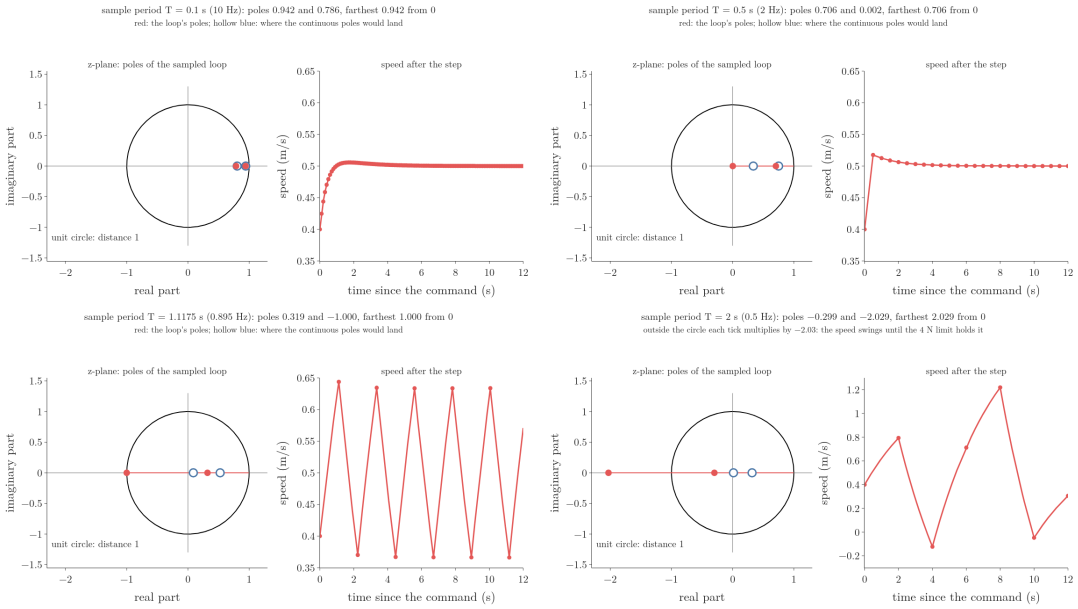


Figure 8: Left: the z-plane, real part across and imaginary part up, with the unit circle; the two poles of the sampled speed loop (red dots, their paths so far as red lines on the real axis) and two hollow blue circles where the continuous loop's poles, −0.575 and −2.175 rad/s, would land through z = eˆ(sT). Right: the speed after the command step, sampled every T; from T = 1.2 s on the speed axis zooms out, because the swings outgrow it. One sample period per frame: T = 0.01 s (poles 0.994 and 0.978), 0.1 s (0.942 and 0.786), 0.5 s (0.706 and 0.002), 1.1175 s (one pole at −1) and 2 s (−0.299 and −2.029, outside the circle; the speed swings until the 4 N limit holds it)

For short sample periods the red poles sit on the hollow circles: at 100 Hz they are 0.994 and 0.978, the continuous poles moved through $z = e^{sT}$. As the period grows they drift away from them, and that drift is the sampling's damage. Tustin's substitution of Section 5.2 has a useful property here: it sends every stable continuous pole inside the unit circle, whatever the sample period (NPTEL DC M3 L2), which is why the Tustin filter of Figure 7 stayed stable. With the loop's poles in hand, we can choose the rate.

6. Choosing the sample rate

> **Key point:** Sample fast enough that half a sample period is a small part of the loop's own timing. A common rule asks for a sample period below 1/(30 × bandwidth), 0.012 s for our speed loop, so about 83 Hz or faster; robot controllers run at hundreds to thousands of Hz, and our 100 Hz loop behaves like the formula.

The opening asked how often the code must run. Sections 3 to 5 give the reasons a slow rate hurts: the hold's half-sample delay eats phase margin, and the loop's poles drift from where the continuous design put them. The table collects the speed loop's numbers (notebook):

Rate	T	Half a sample	Phase lost	Overshoot	Pole distance
100 Hz	0.01 s	0.005 s	0.7°	5.0 %	0.994
10 Hz	0.1 s	0.05 s	7.3°	5.7 %	0.942
2 Hz	0.5 s	0.25 s	36.3°	17.5 %	0.706
0.5 Hz	2 s	1 s	more than the margin	never settles	2.029

The continuous formula overshoots by 4.9 percent. "Pole distance" is how far the farther pole lies from the centre of the z-plane (Section 5.3); above 1 the loop is unstable.

A rule from the loop's speed. How fast is fast enough depends on how fast the loop itself is, measured by its bandwidth (G-3029), the rate of command change at which the loop's response has fallen to 0.707 of its slow-command value. Our speed loop's bandwidth is 2.764 rad/s, about 0.44 Hz (notebook). The CTM digital tutorial asks for a sample period below 1/(30 × bandwidth):

- T, the sample period, in s;
- ω_{BW}, the closed loop's bandwidth, 2.764 rad/s.

$$T < 1/(30 \times \omega_{BW})$$

$$= 1/(30 \times 2.764)$$

$$= 1/82.92 = 0.012 \text{ s}$$

So the loop should tick at least about 83 times a second. Our 100 Hz meets the rule. The 10 Hz loop breaks it, yet its overshoot is only 0.8 points above the formula's, so the rule is cautious. At 2 Hz the loop is far outside the rule and its overshoot has more than tripled.

What real robots do. Robot controllers sample their sensors and update their actuators at hundreds to a few thousands of Hz; faster is of little use given the time constants of the robot and its surroundings (MR §11.1). Faster is not free: each tick costs computing time, and PX4 warns that a gyro rate of 4 kHz or more needs a fast processor (PX4 MC filter tuning).

The rule for the area matters only when the rate is slow. Section 4.3's three rules, in the loop (notebook):

Rate	Forward Euler	Backward Euler	Tustin
100 Hz	5.0 %	4.9 %	4.9 %
10 Hz	5.7 %	5.1 %	5.4 %
2 Hz	17.5 %	46.9 %	32.2 %

At 10 Hz backward Euler comes closest to the formula's 4.9 percent, because it puts the new error into the integral at once instead of one tick later, which removes part of the delay. At 2 Hz the same habit backfires. The new error's whole strip, $k_i T = 5$ N per m/s, joins the force at once, so the first push is 25 N per m/s of error instead of 20, and a loop that already overcorrects in one tick (Section 5.3) overcorrects more. Tustin adds half a strip, 22.5 N per m/s, and lands between. At 100 Hz the choice makes no difference. A fast enough rate fixes the first trouble of the opening. The second, the slow false wave of Figure 2, needs one more idea.

7. Aliasing: a fast wave that looks slow

> **Key point:** A wave that repeats more than once every two samples, faster than half the sample rate (the Nyquist frequency), gives the same samples as a slower wave, so the computer cannot tell them apart: the 1.59 Hz wheel ripple read at 2 Hz looks like a 0.41 Hz wave. Sample fast enough, or filter out the fast waves before sampling.

7.1 The wheel that seems to turn backwards

The 2 Hz dots of Figure 2 traced a slow wave that is not in the reading. The wheel itself shows why. The wheel turns at 10 rad/s. Read at 10 Hz, it has turned 1 rad, 57.3°, between two readings; read at 2 Hz, it has turned 5 rad, 286.5°, between two readings:

$$286.5^\circ - 360^\circ = -73.5^\circ$$

A turn of 286.5° forward lands the wheel's mark exactly where a turn of 73.5° backwards would. The computer sees only where the mark is, so it cannot tell the two apart, and the smaller, backward turn is what the samples look like. Figure 9 plays both. Watch the red wheel on the right jump backwards by 73.5° at every 2 Hz tick while the real wheel on the left spins forwards.

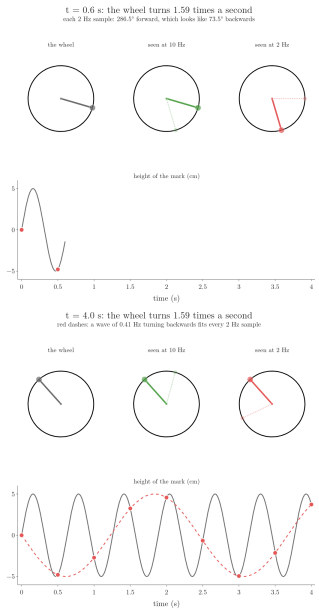


Figure 9: A mark on a wheel turning at 10 rad/s (1.59 turns a second, the robot at 0.5 m/s), one frame per 0.1 s. Top: the wheel (grey), what a computer reading at 10 Hz last saw (green; 57.3° forward between readings, faint: the reading before) and what one reading at 2 Hz last saw (red; 286.5° forward, which looks like 73.5° backwards). Bottom: the mark's height in cm (grey) and the 2 Hz samples (red dots). The last frame adds a slow wave of 0.41 Hz turning backwards (red dashes) that passes through every 2 Hz sample

When samples cannot tell a fast wave from a slower one, the fast wave passes for the slow one. Sampling that makes different signals look the same is **aliasing** (G-3116) (NPTEL DC M1 L1).

Why the false wave is exactly 0.41 Hz. The mark's height, as a fraction of the radius, is $\sin(10t)$ at time t. The 2 Hz ticks come at $t = 0$, 0.5, 1 s and so on, that is at 0.5 s times the tick number k. Subtracting a full turn, 2π, per tick changes no sample:

$$\sin(10 \times 0.5k) = \sin(5k)$$

$$= \sin(5k - 2\pi k)$$

$$= \sin(-1.283\,k)$$

$$= \sin(-2.566 \times 0.5k)$$

So the samples are those of a wave turning backwards at 2.566 rad/s:

$$2.566/(2\pi) = 0.41 \text{ Hz}$$

The same for any wave at angular rate ω read every T seconds:

$$\sin(\omega kT) = \sin\Big((\omega - 2\pi/T)\, kT\Big)$$

In hertz, the wave at f and the wave at $f - f_s$ give identical samples, with f the wave's rate and f_s the sample rate:

$$f - f_s = 1.59 - 2 = -0.41 \text{ Hz}$$

The minus sign is the backward turn.

Which waves are safe. A wave that turns less than half a turn between samples is seen turning the right way, as at 10 Hz. A wave that turns more than half a turn is mistaken for a slower one going the nearer way round. Half a turn per sample means a wave at half the sample rate. With:

- f_s, the sample rate, in Hz;
- f_N, the fastest wave that sampling shows as it is, in Hz.

$$f_N = f_s/2$$

The rate f_N is the **Nyquist frequency** (G-3117) (FBS §8.5). For the 1.59 Hz ripple:

- at 2 Hz, $f_N = 1$ Hz: the ripple is faster, so it is aliased;
- at 10 Hz, $f_N = 5$ Hz: the ripple is seen as it is;
- at 100 Hz, $f_N = 50$ Hz: the ripple is seen as it is.

7.2 What aliasing does to the loop, and the anti-aliasing filter

A false wave in the reading becomes a real motion as soon as the controller acts on it. Figure 10 runs the speed loop at 100 Hz and at 2 Hz while the robot cruises at 0.5 m/s with the rippling reading. Watch the thick lines, the robot's true speed: at 100 Hz it barely moves; at 2 Hz it surges slowly up and down.

The numbers (notebook): at 100 Hz the true speed wobbles by 0.0025 m/s, at 10 Hz by 0.0028 m/s, at 2 Hz by 0.0119 m/s. At 1.59 Hz the controller's pushes come and go too fast for the 8 kg robot to follow them far. At 0.41 Hz each push lasts over a second, and the robot follows. The controller has turned a false reading into a real surge nearly five times larger.

The fix: filter before sampling. A low-pass filter placed before the A/D converter removes the waves above the Nyquist frequency while they are still waves, so they cannot fold onto slow ones. Such a filter is an **anti-aliasing filter** (G-3118) (FBS §8.5). The bottom row of

Figure 10 adds one, with a time constant of 0.5 s, to the 2 Hz loop: its samples now barely move, and the surge falls to 0.0036 m/s.

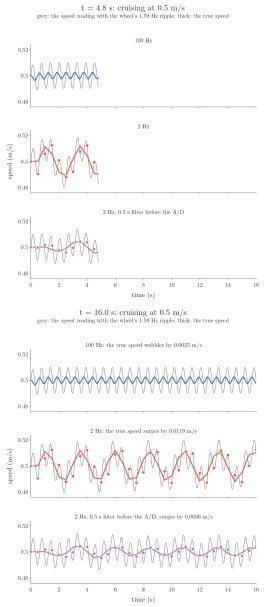


Figure 10: The speed loop cruising at 0.5 m/s while its speed reading carries the wheel's 0.01 m/s ripple at 1.59 Hz (grey), one frame per 0.4 s. Top: at 100 Hz the controller sees the ripple as it is; the robot's true speed (blue) wobbles by 0.0025 m/s. Middle: at 2 Hz the samples (red dots) trace a slow 0.41 Hz wave, the controller chases it, and the true speed (red) surges by 0.0119 m/s. Bottom: at 2 Hz with a low-pass filter of time constant 0.5 s before the A/D, the samples (purple dots) barely move and the true speed (purple) surges by only 0.0036 m/s

At 2 Hz the filter must remove 1.59 Hz while passing what the loop needs, below 1 Hz, and that costs delay (notebook):

Rate and filter before the A/D	Surge	Step overshoot
2 Hz, no filter	0.0119 m/s	17.5 %
2 Hz, filter with time constant 0.2 s	0.0096 m/s	61.9 %
2 Hz, filter with time constant 0.5 s	0.0036 m/s	98.8 %
100 Hz, filter with time constant 0.005 s	0.0025 m/s	5.0 %

A filter slow enough to remove the ripple at 2 Hz adds so much delay that the step overshoot grows more than five times. At 100 Hz the ripple is far below the 50 Hz Nyquist frequency, so the filter only needs to remove much faster noise, and a fast filter costs almost nothing. Sampling fast and filtering early is what PX4 does with its gyro: it samples and filters at the sensor's full raw rate, commonly 8 kHz, before the rate controller uses the result, and it lists the delay each filter setting costs, about 8 ms at a 30 Hz cutoff and 1.9 ms at 120 Hz (PX4

MC filter tuning). Both troubles of the opening now have a cause and a cure, and the last step is to see what real stacks choose.

8. Where real stacks sample

> **Key point:** A real robot runs each layer at its own rate: the planner slowest (Nav2's controller at 20 Hz), the loops near the motors and sensors fastest (PX4 samples its gyro at the sensor's full rate, commonly 8 kHz), and the PID code takes the measured time step at every call.

The opening question, how often to run the code, has different answers at different layers of a real robot. Each answer below comes from the stack's code or parameter files; their full paths are in Sources:

Stack	Loop	Rate	Where
Apollo	longitudinal control: station and speed PID	100 Hz (`ts: 0.01`)	`controller_conf.pb.txt` of the longitudinal PID controller
Nav2	controller server, the local planner	20 Hz (`controller_frequency: 20.0`)	`nav2_params.yaml`
Nav2	velocity smoother	20 Hz by default; can run at 100 Hz to fill in between planner commands	the smoother's `README.md`
PX4	gyro sampling and filtering	the sensor's full raw rate, commonly 8 kHz	`filter_tuning.md`
ros2_control	`control_toolbox` PID	the `dt` passed at each call	`pid.cpp`

Two lessons from the table:

- **Layers run at different rates.** Nav2's planner decides at 20 Hz, and its README suggests running the velocity smoother at 100 Hz so that the robot base gets about five smoothed commands per planner command (nav2_velocity_smoother README). The inner loops near the motors and sensors run fastest, as cascade control (G-3086), a fast inner loop under a slower outer one, needs.
- **The sample period is part of the formula.** ros2_control's `Pid::compute_command` and Apollo's `PIDController::Control` both multiply the error by the measured `dt` at every call (Section 4.2), so a tick that comes late is still integrated with its true length.

9. Summary

Idea	What it is	Our numbers
Sampling	reading a signal only at the clock's ticks	every 0.01 s, 100 Hz
A/D, D/A converters	voltage to number on the way in, number to voltage on the way out	one reading and one force per tick
Zero-order hold	keeps each command until the next tick	a delay of half a sample: 0.25 s at 2 Hz
Difference equation	the next value from the present and past values	$I_{k+1} = I_k + 10\,T\,e_k$
Forward Euler, backward Euler, Tustin	three ways to add up the area under the error	0.0593, 0.0072, 0.0333 m against 0.0281 m
z-transform	z^{-1} means one tick earlier	Tustin PI at 10 Hz: $u_k = u_{k-1} + 20.5\,e_k - 19.5\,e_{k-1}$
Unit circle	stable exactly when every pole is inside	farther pole 0.994 at 100 Hz, 0.706 at 2 Hz, 2.029 at 0.5 Hz
Sample-rate rule	T below $1/(30 \times \text{bandwidth})$	0.012 s, about 83 Hz or more
Aliasing	a wave faster than $f_s/2$ gives the same samples as a slower one	1.59 Hz seen as 0.41 Hz at 2 Hz
Anti-aliasing filter	a low-pass filter before the A/D	at 100 Hz almost free; at 2 Hz overshoot 98.8 %

- The opening question has its answer: written as a running sum and run at 100 Hz, the PI code overshoots by 5.0 percent against the formula's 4.9, because half a tick, 0.005 s, costs only 0.7° of the 84.5° phase margin; at 2 Hz the 0.25 s half-tick costs 36.3°, so the overshoot more than triples to 17.5 percent.
- A computer sees the robot only at its ticks and holds its command in between, so every digital loop carries a delay of about half a sample period, which grows as the rate falls.
- The integral becomes a running sum, a difference equation, so the controller needs only a multiplication and an addition per tick.
- Forward Euler is enough for the integral but blows up a filter whose time constant is shorter than half a sample period (a gap multiplied by -4 each tick at 10 Hz), so fast filters use backward Euler or Tustin, which stay stable at any rate.
- Writing z^{-1} for one tick earlier turns difference equations into algebra, so the whole sampled loop has poles we can compute: at 2 Hz the robot's and the controller's equations combine into one quadratic whose roots, 0.706 and 0.002, lie inside the unit circle.
- The poles slide left as the period grows, because the proportional push held for a whole tick overcorrects more and more, so one pole leaves the unit circle through -1 at 0.89 Hz and the loop is unstable below that rate.
- The CTM rule asks for a period below $1/(30 \times \text{bandwidth})$, so our loop with a bandwidth

of 2.764 rad/s needs about 83 Hz or more, and 100 Hz passes; robots run at hundreds to thousands of Hz.
- A wave faster than half the sample rate gives the same samples as a slower one, so at 2 Hz the wheel's 1.59 Hz ripple becomes a 0.41 Hz false wave, and the loop turns it into a real surge nearly five times the size it has at 100 Hz.
- Filtering before sampling stops aliasing but adds delay, so the practical cure is to sample fast and filter early, as PX4 does at the gyro's full rate.

The robot now runs its speed loop as code at a rate that behaves like the formula, and its readings are free of false waves. Still missing: everything so far holds one speed or one heading at a time. Driving to a goal point and following a planned path, steering back whenever the robot drifts off it, is path tracking, the next chapter, which starts with driving to a point.

10. Sources

Built from

- MATLAB, "4 Ways to Implement a Transfer Function in Code | Control Systems in Practice, Part 7", https://www.mathworks.com/videos/control-systems-in-practice-part-7-4-ways-to-implement-a-transfer-function-in-code-1577967319355.html
- Åström, K. J. and Murray, R. M. (2021). *Feedback Systems*, 2nd ed., §8.5 "State Space Controller Design" (Computer Implementation, Fig. 8.14), §11.5 "Implementation" (Computer Implementation, eqs. 11.16–11.18). Free: https://fbswiki.org (FBS)
- University of Michigan, Control Tutorials for MATLAB and Simulink, "Introduction: Digital Controller Design" (zero-order hold equivalence, the sample-period rule, z-plane stability), ctms.engin.umich.edu, Introduction: Digital Controller Design; read in the SFU mirror, whose `c2dm` is today's `c2d` (CTM digital tutorial)
- Kar, I. and Majhi, S. (IIT Guwahati), NPTEL *Digital Control System*, course 108103008: Module 1 Lectures 1 "Introduction", 2 "Discrete time system representation", 4 "Data Reconstruction"; Module 2 Lecture 1 "Motivation for using Z-transform"; Module 3 Lectures 1 "Stability Analysis of closed loop system in z-plane" and 2 "Stability Analysis using Bilinear Transformation", archive.nptel.ac.in, course 108103008 (NPTEL DC M1 L1 to M3 L2)
- Lynch, K. M. and Park, F. C. (2017). *Modern Robotics: Mechanics, Planning, and Control*. Cambridge University Press. §11.1 "Control System Overview". Free preprint: http://modernrobotics.org (MR §11.1)
- PX4 Autopilot documentation, "MC Filter Tuning & Control Latency", filter_tuning.md in the PX4-Autopilot docs (PX4 MC filter tuning)
- Code and parameter files read for Sections 4.2 and 8:
  - ros2_control: control_toolbox/src/pid.cpp in ros-controls/control_toolbox
  - Apollo: modules/control/control_component/controller_task_base/common/pid_controller.cc in ApolloAuto/apollo
  - Apollo: modules/control/controllers/lon_based_pid_controller/conf/controller_conf.pb.txt in ApolloAuto/apollo
  - Nav2: nav2_bringup/params/nav2_params.yaml in ros-navigation/navigation2
  - Nav2: nav2_velocity_smoother/README.md in ros-navigation/navigation2

(nav2_velocity_smoother README)

Other references

- Brian Douglas, "Discrete control #1: Introduction and overview", YouTube, https://www.youtube.com/watch?v=14cMhrp5wlk
- Brian Douglas, "Discrete control #2: Discretize! Going from continuous to discrete domain", YouTube, https://www.youtube.com/watch?v=rL_1oWrOplk
- Brian Douglas, "Discrete control #3: Designing for the zero-order hold", YouTube, https://www.youtube.com/watch?v=yetLPW9sQaI
- MATLAB, "Important PID Concepts | Understanding PID Control, Part 7", YouTube, https://www.youtube.com/watch?v=tbgV6caAVcs
- SciPy documentation, `scipy.signal.cont2discrete`, https://docs.scipy.org/doc/scipy/reference/ger

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Sampling (G-3105)	Taking a signal's values only at regular instants, the ticks of a clock, such as reading a robot's speed every 0.01 s; it is how a computer sees the world, and between samples it knows nothing.
z-transform (G-3114)	A way of writing a sampled sequence $f_0, f_1, \ldots$ as one expression $F(z) = f_0 + f_1 z^{-1} + \ldots$, so that a delay of one sample is a multiplication by z^{-1}; it turns difference equations into algebra with transfer functions and poles in z.
Aliasing (G-3116)	Sampling making a fast wave give the same samples as a slower one, so the two cannot be told apart, such as a 1.59 Hz ripple read at 2 Hz looking like a 0.41 Hz wave; a controller that acts on the false wave moves the robot for real.
Sample period and sample rate (G-3106)	The time T between two samples, and its inverse, the number of samples per second $f_s = 1/T$ in hertz, such as 0.01 s and 100 Hz; they set how often a digital controller sees the robot and acts.

Term	Meaning
Analog-to-digital (A/D) converter (G-3107)	The device that turns a sensor's voltage into a number at each clock tick, such as a gyro's rate signal; it lets the computer read the robot, one number per tick.
Digital-to-analog (D/A) converter (G-3108)	The device that turns the computer's command, a number, back into a voltage for the actuator at each tick; with a hold it gives the motors a force between ticks.
Zero-order hold (G-3109)	Keeping each command constant from its tick until the next one, so the actuator gets a staircase; simple and cheap, but the held value is on average half a sample period old, so it acts like a delay of $T/2$.
Difference equation (G-3110)	An equation that gives a sequence's next value from its present and past values, such as $I_{k+1} = I_k + k_i T e_k$ for an integral term; it is the form of a controller a computer can run with one multiplication and one addition per tick.
Discretisation (G-3111)	Turning a continuous-time controller or model, written with derivatives and integrals, into difference equations that run once per sample period, by forward Euler, backward Euler, Tustin or the zero-order-hold equivalent; it is the step from a formula to code.
Backward Euler (G-3112)	A discretisation rule that adds up each strip of an integral with the height of its right (newest) sample, $A_{k+1} = A_k + T e_{k+1}$; it stays stable at any sample period, so it suits fast filters inside a controller.
Tustin's method (bilinear transform) (G-3113)	A discretisation rule that adds up each strip of an integral as a trapezoid joining its two samples, which amounts to replacing s by $(2/T)(z-1)/(z+1)$; it is the most accurate of the three simple rules and sends every stable continuous pole inside the unit circle.
Unit circle (z-plane stability) (G-3115)	The circle of radius 1 around the centre of the z-plane; a sampled loop is stable exactly when all its poles lie inside it, because each pole's distance from the centre is the factor its motion is multiplied by at every tick.

Term	Meaning
Nyquist frequency (G-3117)	Half the sample rate, $f_s/2$, such as 50 Hz for a 100 Hz loop; waves slower than it are seen as they are, faster ones are aliased onto slower ones.
Anti-aliasing filter (G-3118)	A low-pass filter placed before the A/D converter that removes waves faster than the Nyquist frequency before they are sampled, so they cannot turn into false slow waves; at a low sample rate it must be slow and adds delay to the loop.
PID controller (G-2965)	A controller whose command is the sum of a proportional, an integral and a derivative part of the error, K_p e + K_i (integral of e) + K_d de/dt; the commonest feedback controller, it reacts to the error now, its past and its trend; with K_d = 0 it is a PI controller, with K_i = 0 a PD controller.
Proportional control (P control) (G-2958)	Feedback that sets the command in proportion to the control error, such as a torque of 0.2 N m per rad of heading error; it pulls the output towards the reference like a spring, but alone it swings when nothing damps the motion and leaves an error under a steady load.
Integral action (G-2963)	A part of the command in proportion to the error's running sum (its integral over time); the sum grows while any error remains, so it removes the error a steady load leaves, at the cost of overshoot and, if too large, instability.
Anti-windup (G-3078)	Any change to a controller with integral action that keeps the integral from growing while the actuator is saturated, so the command leaves the limit as soon as the error changes sign; clamping and back-calculation are the two common kinds.
Pole (G-3000)	A value of s where a transfer function's denominator is zero, so the fraction blows up there, such as -2 (twice) for the PD heading loop; each pole adds a term e^(pt) to every reply, so its real part says fade or grow and its imaginary part how fast it swings.

Term	Meaning
Overshoot (step response) (G-2983)	How far a step response's peak goes past its final value, as a percentage of the final value, such as 52.7 % when a 0.5 rad turn peaks at 0.763 rad; it says how far a reply swings past its target.
Rolling without slipping (G-2290)	A wheel moving so that its contact point does not skid: it moves forward by its radius times the angle it turns and never slides sideways, which is what lets wheel turns predict motion.
Phase (phase shift) (G-3022)	How far the output sine is shifted against the input sine at one rate, as a fraction of a period in degrees, negative when the output is late, such as $-14.9°$ for a 0.261 s lag in a 6.28 s period; it says how late a system answers a wave.
Time delay (dead time) (G-3012)	A wait of T seconds between a signal and its effect, such as a camera reporting the heading late, with transfer function e^(−sT); it has no poles or zeros but limits the gains a loop can use, here making the turn swing for ever near 0.32 s.
Phase margin (G-3045)	How much extra phase lag the loop can take at its gain crossover frequency before the closed loop becomes unstable, $180° + \angle L(j\omega_{gc})$; typical designs keep 30° to 60°.
Gain crossover frequency ω_{gc} (G-3042)	The frequency at which the loop transfer function's gain is 1 (0 dB), such as 2.08 rad/s for the robot's heading loop; the phase margin and the delay margin are read there.
Euler step (G-2299)	One step of predicting motion: hold the rates fixed for a short time Δt and add rate times Δt to each quantity; repeated, it traces a path, more accurately the smaller Δt is.
Derivative filter (G-3101)	A first-order low-pass on the PID's derivative path with time constant $T_f = T_d/N$, N from 5 to 20; it smooths sensor noise such as encoder ticks before the derivative amplifies it, at the cost of some phase margin.

Term	Meaning
Transfer function (control) (G-2999)	The number a linear system multiplies an exponential input e^(st) by, written as a function of s, such as G(s) = 1/(8s + 2) for the robot's speed and 10/s² for its heading; it turns a differential equation into a fraction, so systems can be combined and read by algebra.
Time constant (G-2988)	The time a first-order system takes to close 63 % of the gap to its final value, m/(K_v + b) = 1 s for the robot's speed loop; it fixes the rise time (about 2.2 time constants) and the 2 % settling time (about 4).
Bandwidth (of a system) (G-3029)	The rate at which a system's gain has fallen to 0.707 of its zero-frequency gain (−3 dB), such as 3.02 rad/s for the first-try heading loop (Kp 0.4, Kd 0.08); it marks the fastest input the system still follows.
Cascade control (G-3086)	Nesting feedback loops so that the outer loop's output is the inner loop's reference, such as a heading loop that commands the yaw rate for a gyro loop; it lets each loop be designed in turn, catches disturbances early and offers a place for limits.